

Programare Orientată Pe Obiecte

Curs 13: Fire de lucru (Threads) în Java

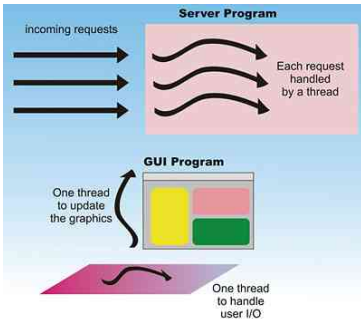
Ianuarie 2026

Introducere

Ce este Multithreading?

- Multithreading este un proces de execuție a mai multor fire simultan.
- Fiecare fir este un subproces ușor.
- Este utilizat pentru a realiza așa numitul thread-based multitasking în cadrul unui singur program.
- Thread based multitasking:
 - Executarea simultană a mai multor sarcini în care fiecare sarcină este o parte independentă separată a aceluiași program.
 - Aceste părți individuale separate ale aceluiași program se numesc Thread-uri (fire de execuție).
- Threadurile sunt fire de lucru paralele care rulează înăuntrul unui program; Ele partajează o zonă de memorie comună.
- Multithreading în Java se referă la un program care execută simultan mai multe threaduri.

Utilitatea firelor de lucru



- Acolo unde trebuie să se întâmple mai multe lucruri simultan:
 - D.e., o aplicație multimedia poate necesita ca procesele audio, video și de control să se execute în paralel
 - Există adesea perioade de așteptare a răspunsului sistemelor de I/E mai lente, timp în care procesorul poate face altceva
 - Programe cum sunt sistemele server/client sunt mult mai ușor de proiectat și scris folosind fire de lucru

Pe sistemele multi-procesor, mașinile virtuale Java pot rula firele de lucru pe procesoare diferite și pot obține astfel prelucrare paralelă adevărată și creșteri de performanțe semnificative față de platformele mono-procesor

Avantajele Multithreading-ului

- Utilizarea eficientă a procesorului.
- Performanță îmbunătățită prin realizarea mai multor sarcini în paralel.
- Simplificarea aplicațiilor complexe.
- Permite comportamentul asincron.

Proprietățile firelor de lucru în Java

- 1 Fiecare fir de lucru își începe execuția la o locație bine cunoscută, predefinită
- 2 Fiecare fir de lucru își execută codul începând de la locația de start, într-o secvență ordonată, predefinită (pentru un set de date de intrare dat)
- 3 Fiecare fir de lucru își execută codul independent de celelalte fire de lucru din program
- 4 Firele de lucru par a avea un anumit grad de simultaneitate în execuție
- 5 Firele de lucru au acces la diferite tipuri de date

Crearea Firelor

Definirea si instantierea unui Thread in JAVA

Un fir de lucru (thread) în Java se poate crea in doua moduri:

- 1 **Extinde Clasa Thread:** Extinderea directă a clasei Thread pentru a crea fire.
- 2 **Implementează Interfața Runnable:** Implementarea interfeței Runnable pentru definirea firelor.

1: Extinde Clasa Thread

Motivul pentru care are sens să se extindă clasa **Thread** este când se dorește realizarea unei versiuni mai specializate a clasei Thread.

Modul cel mai simplu de definire a codului de rulat într-un thread separat este prin:

- Extinderea clasei Thread
- Suprascrierea metodei run()

```
class MyThread extends Thread {  
    public void run() {  
        System.out.println("Firul este in executie");  
    }  
}  
public class Test {  
    public static void main(String[] args) {  
        MyThread t1 = new MyThread();  
        t1.start();  
    }  
}
```

Limitarea acestei abordări este că noua clasă nu va mai putea extinde vreo altă clasă din moment ce a extins clasa Thread

2: Implementează Interfața Runnable

În restul cazurilor (majoritatea cazurilor) când se dorește doar să se specifice ce anume trebuie să execute threadul, se definește o clasă care implementează interfața **Runnable**

- Această variantă oferă flexibilitatea de a extinde orice altă clasă, păstrându-și proprietatea de a putea fi executată într-un fir de lucru separat

```
class MyRunnable implements Runnable {  
    public void run() {  
        System.out.println("Firul este in executie");  
    }  
}  
  
public class Test {  
    public static void main(String[] args) {  
        MyRunnable r = new MyRunnable();  
        Thread t1 = new Thread(r);  
        t1.start();  
    }  
}
```

Indiferent de modalitatea aleasă, rezultă cod ce poate fi executat într-un fir de lucru separat

Exemple

```
class Multithread extends Thread{
    public void run(){
        try{ System.out.println("Thread " + this.getId() + "ruleaza!"); }
        catch(Exception e){ System.out.println("Exceptie prinsa !"); }
    }
}

class MyRunnable implements Runnable{
    public void run() {
        try{ System.out.println("Fir:"+Thread.currentThread().getId()); }
        catch(Exception e){ System.out.println("Exceptie prinsa !"); }
    }
}

public class Main {
    public static void main(String[] args) {
        for (int i = 0; i < 5; i++){
            Multithread obj = new Multithread();
            obj.start();
        }
        for (int i = 0; i < 3; i++){
            Thread obj2 = new Thread(new MyRunnable());
            obj2.start();
        }
    }
}
```

Metoda getId()

- Returnează identificatorul (ID) thread-ului
- ID este un număr pozitiv (long) generat la crearea thread-ului
- ID este unic și rămâne neschimbat pe durata vieții thread-ului.
- Când un thread este terminat, același ID poate fi reutilizat

Instanțierea unui Thread

Fiecare thread începe prin instanțierea unui obiect de clasă Thread:

- Indiferent cum s-a implementat metoda `run()`, prin extinderea clasei Thread sau prin implementarea interfeței Runnable, este nevoie de un obiect de tipul Thread care să facă treaba

- ➊ Pentru varianta când s-a extins clasa Thread, instanțierea este simplă:

```
MyThread t = new MyThread();
```

- ➋ Pentru varianta când s-a implementat interfața Runnable, este nevoie de următorii pași:

```
MyRunnable r = new MyRunnable();  
Thread t = new Thread(r);
```

- ➌ În acest caz, obiectul Runnable este dat ca și argument al constructorului Thread pentru a ști unde se află implementarea metodei `run()`

Instanțierea unui Thread(2)

Același obiect Runnable poate fi pasat ca argument la mai multe threaduri, de exemplu:

```
public class TestThreads {  
    public static void main (String [] args) {  
        MyRunnable r = new MyRunnable();  
        Thread foo = new Thread(r);  
        Thread bar = new Thread(r);  
        Thread wee = new Thread(r);  
    }  
}
```

- Făcând astfel, mai multe threaduri vor rezolva simultan aceeași problemă
- Până aici avem o instanță thread și știm care metodă run() se va executa, dar nu avem încă o execuție
- Pentru execuție e nevoie de invocarea metodei **start()**
- În acest caz, obiectul Runnable este dat ca și argument al constructorului Thread pentru a ști unde

Începerea unui thread

Pentru începerea unui thread se apelează: **t.start()**;

- În acest moment se creează și o nouă stivă de apeluri asociată threadului t
- Doar din momentul apelului metodei start() threadul este considerat în viață (alive), chiar dacă metoda run() este posibil să nu se fi executat încă
- Un thread este considerat mort (no longer alive) după ce metoda run() și-a terminat execuția
- Pentru a verifica starea unui thread (dacă metoda run() și-a terminat sau nu execuția) se poate folosi metoda: t.isAlive()

Ce se întâmplă concret la apelul metodei start():

- Un nou thread începe, având o stivă nouă de apeluri (fiecare thread are stiva proprie de apeluri)
- Starea thread-ului se schimbă de la new la runnable
- Când thread-ul prinde ocazia să se execute, metoda run() se va rula

```
class FooRunnable implements Runnable {
    public void run() {
        for(int x = 0; x < 3; x++)
            System.out.println("Runnable running");
    }
}

public class TestThreads {
    public static void main (String [] args) {
        FooRunnable r = new FooRunnable();
        Thread t = new Thread(r);
        t.start();
    } }
```

Rezultate afisate:

```
Runnable running
Runnable running
Runnable running
```

Începerea unui thread

- Pentru a ști care thread se execută, putem interoga apelând metoda `getName()` din clasa `Thread`
- În următorul exemplu vom da nume threadului și îl vom afișa în metoda `run()` :

```
class NameRunnable implements Runnable {  
    public void run() {  
        System.out.println("NameRunnable running");  
        System.out.println("Run by " + Thread.currentThread().getName());  
    }  
}  
  
public class NameThread {  
    public static void main (String [] args) {  
        NameRunnable nr = new NameRunnable();  
        Thread t = new Thread(nr);  
        t.setName("Rares");  
        t.start();  
    }  
}
```

Rezultate afisate:

NameRunnable running

Run by Rares

Rularea mai multor threaduri

Exemplu cu o singură interfață Runnable și trei fire de lucru:

```
class NameRunnable implements Runnable {  
    public void run() {  
        for (int x = 0; x < 3; x++)  
            System.out.println("Run by" + Thread.currentThread().getName() + ", x  
                is " + x);  
    }  
}
```

```
public class ManyNames {  
    public static void main(String [] args) {  
        NameRunnable nr = new NameRunnable();  
        Thread one = new Thread(nr);  
        Thread two = new Thread(nr);  
        Thread three = new Thread(nr);  
        one.setName("Rares");  
        two.setName("Lucy");  
        three.setName("Ricky");  
        one.start();  
        two.start();  
        three.start();  
    }  
}
```

Rezultate afisate:

```
Run by Lucy, x is 0  
Run by Rares, x is 0  
Run by Ricky, x is 0  
Run by Rares, x is 1  
Run by Rares, x is 2  
Run by Lucy, x is 1  
Run by Ricky, x is 1  
Run by Lucy, x is 2  
Run by Ricky, x is 2
```

Rularea mai multor threaduri

Atenție: aceste rezultate au fost obținute la rularea pe o anumită mașină, dar **această ordine de execuție nu este garantată!**

- Nu există nici o garanție în specificațiile Java că threadurile își vor începe execuția în ordinea în care a fost apelată metoda start()
- Nu există garanția că o dată ce un thread și-a început execuția, o va continua până la final
- Și nici garanția că o buclă se va termina de executat înainte ca un alt thread să înceapă
- Singura garanție este următoarea: Fiecare thread va începe execuția și fiecare thread o va finaliza
- În interiorul unui thread lucrurile se petrec într-o ordine predictibilă, dar acțiunile threadurilor multiple pot fi amestecate într-o ordine neprevăzută
- Dacă același cod se rulează de mai multe ori, sau pe mașini diferite, rezultatul poate să fie diferit

Ciclul de viață al unui thread în Java

- Un fir trece prin mai multe stări în timpul execuției:
 - **New (Nou):** Firul este creat, dar metoda `start()` nu a fost apelată încă.
 - **Runnable:** Metoda `start()` a fost apelată, iar firul este gata să fie executat.
 - **Running (În execuție):** Firul este în execuție activă.
 - **Waiting/blocked/sleeping (În așteptare):** Firul așteaptă o notificare pentru a continua. Instanța este în viață (alive) dar nu e runnable; el poate reveni la starea de runnable mai târziu, dacă, de exemplu un anumit eveniment este interceptat (I/E, etc.)
 - **Timed Waiting:** Firul așteaptă pentru un timp determinat.
 - **Terminated (Terminat):** Firul a terminat execuția.
- Trecerea între stări este gestionată de JVM.

Ordinea execuției thread-urilor

Ordinea Execuției Thread-urilor

- **Planificatorul** ordinii de execuție a threadurilor (Thread Scheduler) ține de JVM:
 - Decide care thread să se execute la un anumit moment de timp (din mulțimea threadurilor eligibile)
 - Scoate threadurile din starea de running la terminarea execuției lor
- O singură stivă de apeluri poate fi executată la un moment dat
- Există un comportament de coadă în planificarea ordinii de execuție, în sensul că o dată ce un thread și-a terminat bucata lui de rulat, este pus în capătul cozii unde își așteaptă din nou rândul pentru execuție
- Chiar dacă nu putem controla ordinea de execuție, există totuși unele unelte pentru a influența această ordine

Metode care ajută la influențarea ordinii de execuție a threadurilor

Din clasa `java.lang.Thread` :

- `sleep()`: Suspendă execuția firului curent pentru o perioadă specificată.
- `yield()`: Sugerează planificatorului să comute la un alt fir gata de execuție.
- `join()`: Așteaptă terminarea execuției unui fir specific.
- `setPriority(int newPriority)`

Din clasa `java.lang.Object`:

- `wait()`: Suspendă firul curent până primește o notificare.
- `notify()`: Trezește un fir în așteptare.
- Utilizare cu blocuri sincronizate:

```
synchronized(obj) { obj.wait(); }  
synchronized(obj) { obj.notify(); }
```

Metoda `sleep()`

- `Thread.sleep()` este o metodă statică din clasa `Thread` care pune în pauză un fir de lucru din interiorul căruia se face invocarea metodei
- Pune în pauză threadul pentru un timp egal cu numărul de milisecunde dat ca argument
- Remarcați că metoda poate fi invocată dintr-un program obișnuit pentru a insera o pauză în singurul fir al programului respectiv
- Pentru o aplicație normală, metoda `run()` din thread corespunde metodei `main()`
- Metoda **`sleep()`** aruncă o excepție de tipul **`InterruptedException`** atunci când un fir "adormit" este întrerupt
 - Interceptează excepția
 - Termină firul de lucru

Exemplu 1 - fără sleep

```
class Numere{  
    public void negative(int k)  {  
        for (int i = -k; i<0; i++) System.out.println("NrNegativ: "+i);  
    }  
    public void pozitive(int k){  
        for (int i = 0; i < k; i++) System.out.println("NrPozitiv: "+i);  
    }  
}
```

```
public class MainSleepE1 {  
    public static void main(String[] args) {  
        int p = 10;  
        Numere obj = new Numere();  
        Thread t1 = new Thread(new Runnable() {  
            public void run() { obj.negative(p); } });  
        Thread t2 = new Thread(new Runnable() {  
            public void run() { obj.pozitive(p); } });  
        t1.start();  
        t2.start();  
        System.out.println("Done!");  
    }  
}
```

Se afiseaza:
Done !
NrNegativ: -5
NrPozitiv: 0
NrNegativ: -4
NrPozitiv: 1
NrNegativ: -3
NrPozitiv: 2
NrNegativ: -2
NrPozitiv: 3
NrNegativ: -1
NrPozitiv: 4

De reținut că ieșirea nu este repetitivă, pentru că nu se știe cu certitudine cât timp durează rularea thread-ului până când este pus în starea de sleep

Exemplu 2: cu sleep()

```
class Numere{
    public void negative(int k)  {
        for (int i = -k; i<0; i++){
            System.out.println("NrNegativ: "+i);
            if (i-k == 2)
                try{ Thread.sleep(1000);}
                catch (InterruptedException e){System.out.println("Except");}
        } }
    public void pozitiv(int k){
        for (int i = 0; i < k; i++) System.out.println("NrPozitiv: "+i);
    } }
```

```
public class MainSleepE1 {
    public static void main(String[] args) {
        int p = 10;
        Numere obj = new Numere();
        Thread t1 = new Thread(new Runnable() {
            public void run() { obj.negative(p); });
        Thread t2 = new Thread(new Runnable() {
            public void run() { obj.pozitiv(p); });
        t1.start();
        t2.start();
        System.out.println("Done!");
    }
```

Se afiseaza:
Done !
NrNegativ: -5
NrPozitiv: 0
NrNegativ: -4
NrPozitiv: 1
NrPozitiv: 2
NrPozitiv: 3
NrPozitiv: 4
NrNegativ: -3
NrNegativ: -2

Terminarea firelor de lucru

- Un fir de lucru se termină la terminarea metodei sale `run()`
- Notificați firul de lucru că ar trebui să își înceteze execuția folosind **`t.interrupt()`**;
- **`interrupt()`** nu face ca firul să se termine - metoda setează un câmp boolean în structura de date a firului
- Java nu forțează terminarea unui fir atunci când acesta este întrerupt
- Este treaba firului de lucru ce anume face atunci când este întrerupt
- Întreruperea reprezintă un mecanism general pentru a "obține atenția" firului de lucru

Terminarea firelor de lucru

Metoda **run()** ar trebui să verifice ocazional dacă firul de lucru a fost întrerupt

- Folosiți metoda `interrupted()`
- Un fir de lucru care a fost întrerupt ar trebui să elibereze resursele pe care le folosește, să "curețe" și să își înceteze execuția

```
public void run()
{
    for(int i=1; i<= REPETITIONS    !Thread.interrupted(); i++)
    {
        //Do work
    }
    //Clean up
}
```

Exemplu pentru interrupt()

Daca un thread este in starea “sleeping” sau “waiting” atunci prin metoda interrupt() se poate intrerupe executia acelui thred si se arunca exceptia InterruptedException

```
class MyClass extends Thread {
public void run() {
    try { for (int i = 0; i < 5; i++) {
        System.out.println("Child Thread executing");
        //Current threads goes to sleeping state;
        //another thread gets the chance to execute
        Thread.sleep(1000);
    } } catch (InterruptedException e){ System.out.print("InterEx");
    } } }
public class TestInterruptThread {
    public static void main(String[] args) throws InterruptedException {
        MyClass thread = new MyClass();
        thread.start();
        // main thread calls interrupt() method on child thread
        thread.interrupt();
        System.out.println("Main thread execution completes"); } }
// Main thread execution completes
//Child Thread executing
//InterruptedException occur
```

Prioritățile threadurilor și yield()

- Pentru a înțelege cum funcționează metoda yield() trebuie să înțelegem mai întâi conceptul de prioritate a threadurilor
- Threadurile se execută într-o anumită ordine (cu o anumită prioritate)
- Prioritatea este de obicei reprezentată printr-un număr de la 1 (minimă) la 10 (maximă)
- Planificatorul de threaduri se bazează în cazul majorității mașinilor virtuale pe o planificare bazată pe priorități, lucru ce implică un fel de secționare temporală (time slicing)
- Prin folosirea secționării temporale, fiecărui thread i se alocă un anumit timp pentru execuție, după care este trimis în starea de runnable pentru a da loc altui thread să se execute

Prioritățile threadurilor și yield()

- Nu toate specificațiile JVM cer implementarea secționării temporale
- Deși multe JVM folosesc secționarea temporală, unele folosesc un planificator de threaduri ce permite unui thread să execute în întregime metoda run()
- Nu vă bazați pe prioritățile threadurilor când proiectați o aplicație multithreading, deoarece comportamentul de planificare bazat pe priorități nu este garantat
- Ex: dacă un thread intră în starea runnable, dar are prioritate mai mare decât threadul curent care rulează, atunci threadul curent este trimis în starea runnable și thread-ul cu prioritatea cea mai mare este ales să ruleze
- Folosiți prioritățile threadurilor ca o modalitate de a îmbunătăți eficiența programului

Setarea priorității unui thread

- Un thread primește implicit o prioritate în momentul în care este creat:

```
MyThread t = new MyThread();  
// prioritatea implicita are valoarea 5
```

- Deoarece threadul main se execută în momentul în care este instanțiat threadul t, acesta va avea aceeași prioritate ca main
- Setarea manuală a priorității unui thread se face folosind metoda **setPriority()** astfel:

```
MyThread t = new MyThread();  
t.setPriority(8);  
t.start();
```

- Valorile priorităților sunt de obicei între 1 (minimă) și 10 (maximă)
- Pentru a verifica exact intervalul permis de JVM folosiți constantele:

```
Thread.MIN_PRIORITY, Thread.NORM_PRIORITY, Thread.MAX_PRIORITY
```

Metoda statică **Thread.yield()**

- Are rolul de a duce threadul curent din stadiul de running în runnable pentru a permite threadurilor cu **aceeași prioritate** să ruleze și ele
- Sugerează planificatorului să cedeze controlul altor fire gata de execuție.
- Nu garantează schimbarea imediată a firului.
- Chiar dacă yield() cauzează trecerea threadului curent în starea runnable, se întâmplă adesea ca același thread să fie ales spre execuție

Exemplu cu Thread.yield()

Analogie: Utilizarea unui ATM

- Imaginați-vă un scenariu în care mai multe persoane încearcă să folosească un bancomat (ATM).
- Dacă o persoană realizează că mai are nevoie de timp pentru a se hotărî ce sumă să retragă, poate să "cedeze" locul unei alte persoane care așteaptă, lăsând-o să folosească ATM-ul între timp.

Exemplu cu Thread.yield()

```
class ATMUser extends Thread {
    public ATMUser(String name) {
        super(name); // Seteaza numele firului
    }
    @Override
    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println(getName() + " foloseste ATM-ul. Tranzactia "
                               + i);
            // Firul cedeaza locul altor fire
            Thread.yield();
            System.out.println(getName() + "continua utilizarea ATM-ului.");
        }
        System.out.println(getName() + " a terminat la ATM.");
    }
}
```

Exemplu cu Thread.yield()

```
public class ExempluYield {  
    public static void main(String[] args) {  
        System.out.println("Simularea utilizarii ATM-ului incepe...");  
        // Crearea firelor (utilizatori ai ATM-ului)  
        ATMUser utilizator1 = new ATMUser("Utilizator 1");  
        ATMUser utilizator2 = new ATMUser("Utilizator 2");  
        ATMUser utilizator3 = new ATMUser("Utilizator 3");  
        // Pornirea firelor  
        utilizator1.start();  
        utilizator2.start();  
        utilizator3.start();  
    }  
}
```

Exemplu cu yield()

- Fiecare thread reprezintă un utilizator al ATM-ului.
- Fiecare utilizator efectuează trei tranzacții
- În timpul fiecărei tranzacții, threadul curent face `Thread.yield()` pentru a ceda controlul execuției altor fire care așteaptă.
- Planificatorul de thread-uri decide care fir preia controlul.
- Acest comportament poate varia în funcție de JVM și de sistemul de operare.
- Firele rulează și în timpul execuției pot ceda locul altor fire, asigurând o partajare echitabilă a resurselor.

Metoda Thread.join()

- Permite unui fir să aștepte terminarea unui alt fir.
- Permite unui thread să se poziționeze la sfârșitul altui thread ("join onto the end")
- Dacă un thread B nu poate să își facă treaba decât după ce threadul A a fost executat, atunci vrem să se facă "join" între threadurile A și B
- Acest lucru înseamnă că threadul B devine runnable după ce threadul A a fost executat

```
Thread t = new Thread();  
t.start();  
t.join(); // sau t.join(500);
```

- Acest cod ia threadul în lucru (în acest caz - main) și face "join" cu threadul t
- Metoda join(500) cu parametru se interpretează astfel: threadul principal (main) așteaptă până când t este finalizat, dar dacă durează mai mult de 500 ms, acesta devine din nou runnable

Exemplu cu Thread.join()

Simulați o aplicație pentru pregătirea unui tort. Pașii necesari pentru realizarea tortului ar fi:

- Pregătire ingrediente
- Coacere blat de tort
- Decorare tort

Aceste sarcini sunt reprezentate ca fire de lucru separate (threads).

Firul principal (main thread) poate "să mănânce tortul" doar atunci când toate aceste sarcini sunt finalizate.

Exemplu cu Thread.join()

```
class Sarcina extends Thread {
    private final String numeSarcina;
    private final int durata;

    public Sarcina(String numeSarcina, int durata) {
        this.numeSarcina = numeSarcina;
        this.durata = durata;
    }

    @Override
    public void run() {
        try {
            System.out.println(numeSarcina + " a inceput.");
            Thread.sleep(durata); // Simuleaza durata necesara pentru
                                   a finaliza sarcina
            System.out.println(numeSarcina + " a fost finalizata.");
        } catch (InterruptedException e) {
            System.out.println(numeSarcina + " a fost intrerupta.");
        }
    }
}
```

Exemplu cu Thread.join()

```
public class ExempluCoacereTort {  
    public static void main(String[] args) {  
        System.out.println("Incepe procesul de coacere a tortului!");  
        // Crearea sarcinilor  
        Sarcina pregatireI = new Sarcina("Pregatire Ingrediente", 2000);  
        Sarcina coacereTort = new Sarcina("Coacerea Tortului", 5000);  
        Sarcina decorareTort = new Sarcina("Decorarea Tortului", 3000);  
        // Pornirea sarcinilor  
        pregatireI.start();  
        coacereTort.start();  
        decorareTort.start();  
        try {  
            // Asteapta finalizarea fiecărei sarcini  
            pregatireI.join(); // Firul principal, main, asteapta pana cand "  
                Pregatirea Ingredientelor" este finalizata  
            coacereTort.join(); // Firul principal, main, asteapta pana cand "  
                Coacerea Tortului" este finalizata  
            decorareTort.join(); // Firul principal, main, asteapta pana cand "  
                Decorarea Tortului" este finalizata  
        } catch (InterruptedException e) {  
            System.out.println("Procesul Coacere Tort a fost intrerupt.");  
        }  
        System.out.println("Tortul este gata! Pofta buna!");  
    }  
}
```

Exemplu cu Thread.join()

- Cele trei thread-uri sunt pornite folosind metoda start().
- **Firul principal folosește join():**
 - pregatirel.join(): Firul principal așteaptă ca sarcina de pregătire să fie finalizată înainte să continue.
 - coacereTort.join(): După ce pregătirea este completă, firul principal așteaptă coacerea.
 - decorareTort.join(): În final, firul principal așteaptă decorarea.
- Firele nu interacționează între ele:
 - Fiecare fir rulează independent. Nu există logică de sincronizare sau dependență între ele. join() este folosit doar de firul principal pentru a controla fluxul execuției.
- Execuție paralelă: cele trei thread-uri rulează în paralel, iar timpul total de execuție depinde de cel mai lung fir (în acest caz Coacerea Tortului)
- join() este util deoarece permite firului principal (main) să controleze ordinea de execuție pas cu pas

Exemple

Exemplu 1:

Se consideră o brutărie cu 3 angajați (thread-uri) care lucrează în paralel:

- Brutar: frământă și coace pâinea (folosește sleep pentru timpul de coacere)
- Casier: procesează clienți (folosește yield pentru a fi "politicos")
- Manager: așteaptă ca brutarul să termine (folosește join)

Cerință

Rulați fiecare exemplu de mai multe ori și observați diferențele în output!

Metoda `sleep()` – Implementare

BrutarTask.java – Simulează coacerea pâinii

```
class BrutarTask implements Runnable {  
    @Override  
    public void run() {  
        try {  
            System.out.println("[Brutar] Incep sa framant aluatul..");  
            Thread.sleep(1000); // 1 secunda de framantat  
  
            System.out.println("[Brutar] Bag painea la cuptor...");  
            Thread.sleep(2000); // 2 secunde de coacere  
  
            System.out.println("[Brutar] Painea e gata!");  
        } catch (InterruptedException e) {  
            System.out.println("[Brutar] Am fost intrerupt!");  
        }  
    }  
}
```

Observație

`sleep()` aruncă `InterruptedException` – trebuie tratată!

Metoda `sleep()` – Utilizare

Pornirea thread-ului

```
public class DemoSleep {  
    public static void main(String[] args) {  
        Thread brutar = new Thread(new BrutarTask());  
        brutar.start();  
        System.out.println("[Main] Brutarul a inceput munca!");  
    }  
}
```

Output (garantat în această ordine relativă):

```
[Main] Brutarul a inceput munca!  
[Brutar] Incep sa framant aluatul...  
    ... (1 secundă) ...  
[Brutar] Bag painea la cuptor...  
    ... (2 secunde) ...  
[Brutar] Painea e gata!
```

Metoda `yield()` – Implementare

Două tipuri de casieri: politicos vs egoist

```
class CasierPoliticos implements Runnable {
    @Override
    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println("[POLITICOS] Servesc clientul #" + i);
            Thread.yield(); // Cedeaza politicos procesorul
        }
    }
}

class CasierEgoist implements Runnable {
    @Override
    public void run() {
        for (int i = 1; i <= 3; i++) {
            System.out.println("[EGOIST] Servesc clientul #" + i);
            // Nu face yield - incearca sa tina procesorul
        }
    }
}
```

Metoda `yield()` – Rezultate Posibile

Pornirea celor doi casieri

```
public class DemoYield {  
    public static void main(String[] args) {  
        Thread t1 = new Thread(new CasierPoliticos());  
        Thread t2 = new Thread(new CasierEgoist());  
        t1.start();  
        t2.start();  
    }  
}
```

Output posibil #1:

```
[POLITICOS] #1 -> [EGOIST] #1 -> [EGOIST] #2 -> ...
```

Output posibil #2:

```
[EGOIST] #1 -> [EGOIST] #2 -> [POLITICOS] #1 -> ...
```

Concluzie

Ordinea este **nedeterministă**! `yield()` NU garantează nimic.

Metoda `yield()` – Observații

Definiție

`Thread.yield()` **sugerează** scheduler-ului că thread-ul curent este dispus să cedeze procesorul.

Caracteristici:

- Este doar o **SUGESTIE** – poate fi ignorată!
- Nu garantează că alt thread va primi procesorul
- Thread-ul rămâne în starea **RUNNABLE**
- Comportament dependent de sistemul de operare

Atenție!

`yield()` este **rar folosit** în practică. Nu vă bazați pe el pentru sincronizare!

Când se folosește (rar):

- Optimizări fine de performanță
- Debugging/testing pentru a provoca race conditions

Metoda `join()` – Task-uri Separate

Brutarul produce, Managerul face inventarul

```
class BrutarProducator implements Runnable {
    @Override
    public void run() {
        try {
            System.out.println("[Brutar] Incep productia...");
            Thread.sleep(2000); // Produce painea
            System.out.println("[Brutar] Am terminat 10 paini!");
        } catch (InterruptedException e) { e.printStackTrace(); }
    }
}

class ManagerInventar implements Runnable {
    private Thread brutar;
    public ManagerInventar(Thread brutar) { this.brutar = brutar; }
    @Override
    public void run() {
        System.out.println("[Manager] Astept sa fie gata painea...");
        try {
            brutar.join(); // BLOCHEAZA pana termina brutarul!
            System.out.println("[Manager] Fac inventarul: 10 paini");
        } catch (InterruptedException e) { e.printStackTrace(); }
    }
}
```

Metoda `join()` – Utilizare și Output

Pornirea thread-urilor

```
public class DemoJoin {  
    public static void main(String[] args) {  
        Thread brutar = new Thread(new BrutarProducator());  
        Thread manager = new Thread(new ManagerInventar(brutar));  
  
        // Pornim INTAI managerul (dar el va astepta)  
        manager.start();  
        brutar.start();  
    }  
}
```

Output (ÎNTOTDEAUNA în această ordine):

```
[Manager] Astept sa fie gata painea...  
[Brutar]  Incep productia...  
        ... (2 secunde) ...  
[Brutar]  Am terminat 10 paini!  
[Manager] Fac inventarul: 10 paini
```

Metoda `join(timeout)` – Așteptare Limitată

Ce facem dacă brutarul e prea lent?

```
public class DemoJoinTimeout {  
    public static void main(String[] args) throws Exception {  
        Thread brutarLent = new Thread(new Runnable() {  
            public void run() {  
                try {  
                    System.out.println("[Brutar] Lucrez 5 secunde..");  
                    Thread.sleep(5000);  
                } catch (InterruptedException e) { }  
            }  
        });  
  
        brutarLent.start();  
        brutarLent.join(2000); // Astept MAXIM 2 secunde  
  
        if (brutarLent.isAlive()) {  
            System.out.println("N-a terminat! Continui fara el.");  
        } else {  
            System.out.println("A terminat la timp!");  
        }  
    }  
}
```

Sincronizare

Sincronizarea codului

- Sincronizarea metodelor previne accesarea simultană a codului dintr-o metodă de mai multe threaduri
- Ce s-ar întâmpla dacă două threaduri ar încerca simultan să seteze starea unui obiect?
- Cuvântul cheie **synchronized** poate fi folosit:
 - Ca modificador al metodei
 - La începutul unui bloc de cod
- În timp ce un singur thread are dreptul de a accesa codul sincronizat al unei instanțe, restul codului (nesincronizat) poate fi accesat de mai multe threaduri
- Când un thread intră în starea sleep codul blocat de el nu va fi accesibil altor threaduri

Sincronizarea codului - exemplu

```
class Table {  
    synchronized void printTable(int n) {  
        for (int i = 1; i <= 5; i++) {  
            System.out.println(n * i);  
            try { Thread.sleep(400); }  
            catch (Exception e) { System.out.println(e); }  
        }  
    }  
}
```

```
class MyThread extends Thread {  
    Table t;  
    MyThread(Table t) { this.t = t; }  
    public void run() { t.printTable(5); }  
}  
  
public class Test {  
    public static void main(String[] args) {  
        Table obj = new Table();  
        MyThread t1 = new MyThread(obj);  
        MyThread t2 = new MyThread(obj);  
        t1.start();  
        t2.start();  
    }  
}
```

5
10
15
20
25
5
10
15
20
25

Același cod fără synchronized!

```
class Table {  
    void printTable(int n) {  
        for (int i = 1; i <= 5; i++) {  
            System.out.println(n * i);  
            try { Thread.sleep(400); }  
            catch (Exception e) { System.out.println(e); }  
        }  
    }  
}
```

```
class MyThread extends Thread {  
    Table t;  
    MyThread(Table t) { this.t = t; }  
    public void run() { t.printTable(5); }  
}  
  
public class Test {  
    public static void main(String[] args) {  
        Table obj = new Table();  
        MyThread t1 = new MyThread(obj);  
        MyThread t2 = new MyThread(obj);  
        t1.start();  
        t2.start();  
    }  
}
```

```
5  
5  
10  
10  
15  
15  
20  
20  
25  
25
```

Sincronizarea codului - exemplu 2

Scriem o aplicație care simulează utilizarea unui cont bancar la care au acces mai multe persoane care încearcă să efectueze simultan tranzacții (retrageri de bani din cont).

- Dacă nu sincronizam accesul la cont pot apărea probleme precum retrageri incorecte (sold negativ)!
- Fiecare persoană poate fi asociată cu un thread separat
- Pentru a ne asigura că soldul rămâne corect, doar o persoană accesează contul la un moment dat, folosim sincronized.
- Firele de lucru sunt pornite simultan dar datorită lui synchronized metoda de extragere este accesată doar de un singur thread la un moment dat.

Sincronizarea codului - exemplu 2

```
class ContBancar {
    private int sold;
    public ContBancar(int soldInitial) { this.sold = soldInitial; }
    // Metoda sincronizata pentru retragere
    public synchronized void retrageBani(String numeClient, int suma) {
        System.out.println(numeClient + " incearca sa retraga " + suma + "
            lei.");
        if (sold >= suma) {
            System.out.println(numeClient + " a reusit retragerea.");
            try { Thread.sleep(1000); // Simuleaza timpul pt procesarea
                tranzactiei
            } catch (InterruptedException e) { System.out.println("Procesare
                intrerupta."); }
            sold -= suma;
            System.out.println(numeClient + " a retras " + suma + " lei.
                Sold ramas: " + sold + " lei.");
        } else {
            System.out.println(numeClient + " nu a reusit retragerea. Sold
                insuficient (" + sold + " lei).");
        }
    }
}
```

Sincronizarea codului - exemplu 2

```
class Client extends Thread {
    private final ContBancar contBancar;
    private final int suma;
    public Client(String nume, ContBancar contBancar, int suma) {
        super(nume);
        this.contBancar = contBancar;
        this.suma = suma;
    }
    public void run() {
        contBancar.retrageBani(getName(), suma);
    } }
public class ExempluSynchronized {
    public static void main(String[] args) {
        ContBancar contBancar = new ContBancar(500);
        Client client1 = new Client("Client 1", contBancar, 300);
        Client client2 = new Client("Client 2", contBancar, 200);
        Client client3 = new Client("Client 3", contBancar, 100);
        client1.start();
        client2.start();
        client3.start();
    } }
```

Sincronizare și blocare (lock)

- Sincronizarea funcționează cu lacăte (locks)
- Fiecare obiect Java are încorporat un singur lock care intervine doar atunci când obiectul deține metode sincronizate
- Doar metode (sau blocuri) pot fi sincronizate, nu și variabilele sau clasele
- Nu toate metodele necesită să fie sincronizate
- O clasă poate avea atât metode sincronizate, cât și nesincronizate
- Dacă două metode încearcă să execute o metodă sincronizată, și ambele folosesc aceeași instanță, threadurile vor putea să execute doar pe rând această metodă

Interacțiunea dintre threaduri

- Un alt lucru de știut despre threaduri este cum interacționează între ele pentru a-și comunica diferite aspecte (ex. lock status)
- Metodele responsabile pentru comunicarea între threaduri sunt: **wait()**, **notify()**, **notifyAll()** din clasa **Object**
- Un exemplu ar fi o aplicație de mail:
 - Aplicația are două threaduri: unul responsabil cu trimiterea mailurilor (T1), iar celălalt cu procesarea lor (T2)
 - T2 trebuie să verifice frecvent dacă există vreun mail de procesat
 - Folosind mecanismul wait-notify, T2 poate verifica existența mailurilor de procesat, și dacă nu găsește nici unul, poate spune astfel: "nu o să-mi irosesc timpul verificând în continuu, ies să fac altceva, iar când T1 (mail delivery) aduce un mail, îmi va trimite o notificare să știu să trec în starea runnable și să îmi fac treaba"

Metodele wait() și notify()

- Metoda wait() lasă threadul să spună: nu este nimic de lucru pentru mine așa că pune-mă în starea waiting, sau adaugă-mă în lista de așteptare
- Metoda notify() este folosită pentru a transmite un semnal către un thread din lista de așteptare a obiectului
- Nu se poate specifica pe care waiting thread să îl notifice
- Metoda notifyAll() trimite semnal tuturor threadurilor în starea waiting
- Metodele wait(), notify(), notifyAll() trebuie să fie apelate dintr-un cod sincronizat!
- Un thread invocă una din aceste metode pe un obiect anume și threadul trebuie să dețină lock-ul acelui obiect.

Exemplu cu wait() și notify()

Ralizăm o aplicație care simulează o cafenea cu un singur barista :).

- 1 Avem un singur barista care preia comenzile
- 2 Avem un client care așteaptă să primească comanda
- 3 Barista va notifica clientul (folosind notify()) când comanda este gata, iar clientul va aștepta folosind wait().

Exemplu cu wait() și notify()

```
// Clasa care reprezinta masa de lucru (resursa partajata)
class MasaDeLucru {
    private boolean comandaGata = false;
    // Metoda pentru a pregati comanda
    public synchronized void pregatesteComanda() {
        System.out.println("Barista: Pregatesc comanda...");
        try { Thread.sleep(2000); // Simuleaza timpul pt pregatire comanda
        } catch (InterruptedException e) {System.out.print("Barista:
            AmFostIntrerupt.");}
        comandaGata = true;
        System.out.println("Barista: Comanda este gata. Notific clientul");
        notify(); // Notifica clientul ca comanda este gata
    }
    // Metoda pentru a ridica comanda
    public synchronized void asteaptaComanda() {
        while (!comandaGata) {
            try { System.out.println("Client: Astept comanda...");
                wait(); // Asteapta pana cand comanda este gata
            } catch (InterruptedException e) {
                System.out.println("Client: Am fost intrerupt.");
            }
        }
        System.out.println("Client:am primit comanda! Plec din cafenea.");
    } }
}
```

Exemplu cu wait() și notify()

```
// Clasa pentru firul care reprezinta barista
class Barista extends Thread {
    private final MasaDeLucru masa;
    public Barista(MasaDeLucru masa) { this.masa = masa; }
    public void run() { masa.pregatesteComanda(); }
}

// Clasa pentru firul care reprezinta clientul
class Client extends Thread {
    private final MasaDeLucru masa;
    public Client(MasaDeLucru masa) { this.masa = masa; }
    public void run() { masa.asteaptaComanda(); }
}

public class ExempluWaitNotify {
    public static void main(String[] args) {
        MasaDeLucru masa = new MasaDeLucru();
        Barista barista = new Barista(masa);
        Client client = new Client(masa);
        client.start();
        barista.start();
    }
}
```

De la wait()/notify() la BlockingQueue

În multe aplicații multi-thread apare modelul **Producer–Consumer**:

- un thread **produce** date (ex: comenzi, clienți, mesaje)
- unul sau mai multe threaduri **consumă** datele și le procesează

Problema: fără un mecanism de sincronizare, consumatorii fie:

- verifică în buclă dacă există elemente (*busy waiting*), consumând CPU
- sau trebuie să implementăm manual `wait()` / `notify()`

Soluție modernă: **BlockingQueue** oferă o coadă thread-safe care **blochează automat** firele când nu există elemente / nu există spațiu.

BlockingQueue (java.util.concurrent)

BlockingQueue = coadă thread-safe pentru comunicare între fire de execuție.

Metode importante:

- `put(x)` — adaugă element; **așteaptă** dacă coada e plină
- `take()` — scoate element; **așteaptă** dacă coada e goală
- `offer(x) / poll()` — variante non-blocking
- `poll(timeout)` — așteaptă până la timeout

Avantaje:

- elimină nevoia de `synchronized + wait()/notify()`
- reduce riscul de erori (race conditions, deadlock, spurious wakeups)
- previne *busy waiting*

BlockingQueue vs wait()/notify()

wait()/notify():

- necesită `synchronized`
- trebuie verificate condiții într-un `while(...)` `wait();`
- `notify()` / `notifyAll()` trebuie apelate corect

BlockingQueue:

- aceeași logică este implementată în bibliotecă
- `take()` înlocuiește `while(empty) wait();`
- `put()` înlocuiește `while(full) wait();`

Concluzie: `BlockingQueue` este o abstracție mai sigură și mai ușor de folosit pentru `Producer-Consumer`.

Exemplu: Cafenea cu BlockingQueue (fără wait/notify)

```
import java.util.concurrent.*;

public class CafeaBlockingQueue {
    static BlockingQueue<String> coada = new ArrayBlockingQueue<>(1);
    static class Barista extends Thread {
        public void run() {
            try {
                System.out.println("Barista: Pregatesc comanda...");
                Thread.sleep(2000);
                coada.put("Cafea"); // pune comanda, poate astepta daca e
                                   plina
                System.out.println("Barista: Comanda pusa pe teigheea.");
            } catch (InterruptedException e) { }
        }
    }

    static class Client extends Thread {
        public void run() {
            try {
                System.out.println("Client: Astept comanda...");
                String comanda = coada.take(); // asteapta daca nu exista
                                                comanda
                System.out.println("Client: Am primit -> " + comanda);
            } catch (InterruptedException e) { }
        }
    }
}
```

Explicații

În exemplul cafelei avem două fire de execuție:

- **Barista** (*producer*) — pregătește comanda
- **Client** (*consumer*) — așteaptă și preia comanda

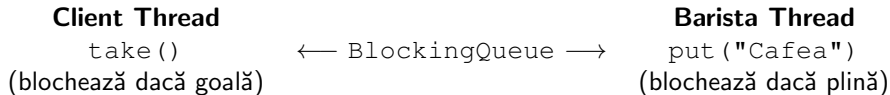
Coada (BlockingQueue) **NU** stochează thread-uri. Coada stochează **date** (ex: comanda).

BlockingQueue este obiectul comun prin care thread-urile comunică:

- Barista pune comanda în coadă
- Clientul scoate comanda din coadă

Analogie: BlockingQueue = tezghea dintre Barista și Client.

BlockingQueue este interfața de comunicare între thread-uri:



Idee: thread-urile nu “se cheamă” direct între ele; ele comunică printr-o resursă sincronizată.

Cum apare legătura dintre BlockingQueue și thread-uri?

Ambele thread-uri folosesc **același obiect** de tip `BlockingQueue`:

```
static BlockingQueue<String> coada = new ArrayBlockingQueue<>(1);
```

Legătura este prin acces concurent la aceeași resursă:

- Clientul apelează: `coada.take()`
- Barista apelează: `coada.put("Cafea")`

Coadă oferă:

- sincronizare automată
- blocare/deblocare automată a firelor
- comunicare între thread-uri fără `synchronized` + `wait()`/`notify()`

Execuție pas cu pas(I)

1) Pornește Clientul primul si execută:

```
String comanda = coada.take();
```

Dar coada este goală $\Rightarrow$ `take()` face:

- thread-ul Client trece automat în starea **WAITING**
- nu consumă CPU (*nu există busy waiting*)
- așteaptă până apare un element în coadă

Deci aici `BlockingQueue` înlocuiește `wait()`.

2) Barista pregătește comanda

După pregătire, Barista execută: `coada.put("Cafea");`

Ce se întâmplă:

- 1 String-ul "Cafea" este adăugat în coadă
- 2 coada **deblochează automat** thread-ul Client blocat în `take()`
- 3 thread-ul Client primește comanda și continuă execuția

Observație: aici, `put()` declanșează comportamentul echivalent cu `notify()`

Comparatie cu wait()/notify() (concluzie de examen)

Cu wait()/notify():

- trebuie `synchronized`
- condiții verificate manual: `while(!gata) wait();`
- după schimbare: `notify()/notifyAll()`

Cu BlockingQueue:

- `take()` $\approx$ **wait până există element**
- `put()` $\approx$ **semnalizare / trezire automată**
- sincronizarea este implementată corect în biblioteca Java

De ce folosim o coadă de capacitate 1?

În exemplu:

```
new ArrayBlockingQueue<>(1)
```

Interpretare: **tejghea cu un singur loc:**

- dacă Barista pune o comandă și Clientul nu o ridică, coada devine plină
- un al doilea `put()` ar bloca Barista până se eliberează loc

Deci `BlockingQueue` poate bloca:

- consumatorul când coada e goală (`take()`)
- producătorul când coada e plină (`put()`)